Software Engineering Project May - 2026

Milestone 5 Deliverables for Neighbourly

Submitted By

Team Name: BugsBunny

Team Code: Team-015



IITM Online BS Degree Program,
Indian Institute of Technology, Madras, Chennai
Tamil Nadu, India, 600036

Apartment Association Management System

Project Details

Software Name: Neighbourly - Apartment Association Management System

Date of Submission: 23-08-2026

Team Details

Team Name: BugsBunny
Team Code: Team-015

Name	Roll Number	Role *
YASH KARSH	22f3002322	Project Architect
Juhi Parekh	23f2005473	Scrum Master
SAFWAN HUMAYUN	24f1001022	Frontend Developer
SANIYA NAAZ	23f2002592	Backend Developer
Saumya Sarkar	21f3001551	Backend Developer/Software Tester

*These are the team's primary roles, and members have collaborated across tasks as the project went on.

Table of Contents

Sl. No.	Topic	Pages
1	Project Overview	4-5
2	User Identification & Requirements	5-7
3	System Design	7-9
4	Implementation	9-12
5	Testing and Validation	12-14
6	User Feedback and Improvements	14-16
7	Code Review and Issue Tracking	16-19
8	Milestones and Sprint Summary	19-25
9	Final System Status and Conclusion	25-26

1. Project Overview

1.1 Introduction

Neighborly is a web-based apartment association management platform designed to digitize and centralize residential community operations. The system provides a unified platform for managing residents, complaints, maintenance tasks, visitors, facility bookings, announcements, notifications, billing, community engagement, and AI-assisted services.

The project was developed incrementally through multiple milestones, beginning with user identification and requirements gathering, followed by system design, backend implementation, testing, user feedback, and feature enhancement. The final system combines role-based access control with structured workflows and AI-powered functionality to improve efficiency, transparency, and convenience for residents and apartment management personnel.

1.2 Problem Statement

Apartment communities often rely on fragmented channels such as WhatsApp groups, phone calls, spreadsheets, and manual records to manage complaints, visitors, maintenance, facility bookings, communication, and payments. These processes can result in poor visibility, delays, data fragmentation, and increased administrative effort.

Neighborly addresses these challenges by providing a centralized platform through which different stakeholders can access and manage apartment services according to their roles.

1.3 Project Objectives

The primary objective of Neighborly is to provide a centralized platform that simplifies and digitizes apartment association operations.

- Centralize apartment association operations on a single platform.
- Provide structured complaint and maintenance management.
- Simplify visitor registration and verification.
- Enable convenient facility booking and conflict detection.
- Centralize announcements and notifications.
- Support billing, payment recording, and receipts.
- Enable community polls and resident participation.
- Use AI to assist with complaint classification, staff recommendations, search, reporting, and community interaction.
- Ensure secure and reliable operation through authentication, RBAC, validation, and comprehensive testing.

1.4 Final System Overview

Metric	Final Status
API Endpoints	97
Modules Implemented	10
Test Cases	103
Passing Test Cases	103
Failing Test Cases	0
Unverified Operations	0
User Stories	US-01–US-20
Working Prototype	Completed

---

2. User Identification & Requirements

2.1 Identified Users

The system was designed around the stakeholders identified during the user research phase.

User Type	Role in the System
Residents/Apartment Owners	Raise complaints, book facilities, pre-register visitors, view announcements, notifications and bills, and participate in polls
Tenants	Access resident-facing services according to their assigned permissions
Apartment Admins / Committee	Manage residents, complaints, facilities, announcements, billing, polls and other community operations
Security Guards	Verify visitors and record visitor entry and exit
Maintenance Staff	Receive, accept, schedule, start and complete assigned maintenance tasks

Visitors/Vendors	External users whose visits are managed through the visitor management workflow

2.2 Key User Problems Identified

User research highlighted several recurring problems with existing apartment management practices and communication methods:

- Dependence on WhatsApp groups and informal communication for notices and updates.
- Difficulty tracking complaints and their resolution status.
- Manual visitor registration and coordination at the security gate.
- Lack of a centralized facility-booking system.
- Limited visibility into maintenance tasks and assignments.
- Manual handling of billing and payment-related information.
- Lack of a structured mechanism for community participation and voting.

2.3 Final User Stories

The requirements evolved based on user research and subsequent feedback. The final system covers the original resident, administration, maintenance and security workflows along with additional billing, community and AI capabilities.

US	Requirement	US	Requirement
US-01	Resident views maintenance request status	US-02	Resident reserves shared facilities
US-03	Resident pre-registers visitors	US-04	Resident receives notifications
US-05	Resident views announcements	US-06	Admin assigns maintenance requests
US-07	Admin publishes announcements	US-08	Maintenance staff manages assigned tasks
US-09	Security guard verifies visitor details	US-10	Security guard records visitor entry/exit
US-11	Visitor identity is verified promptly	US-12	Visitor's approved visit details are available to security
US-13	Resident is notified when visitor arrives	US-14	Vendor service visit is logged

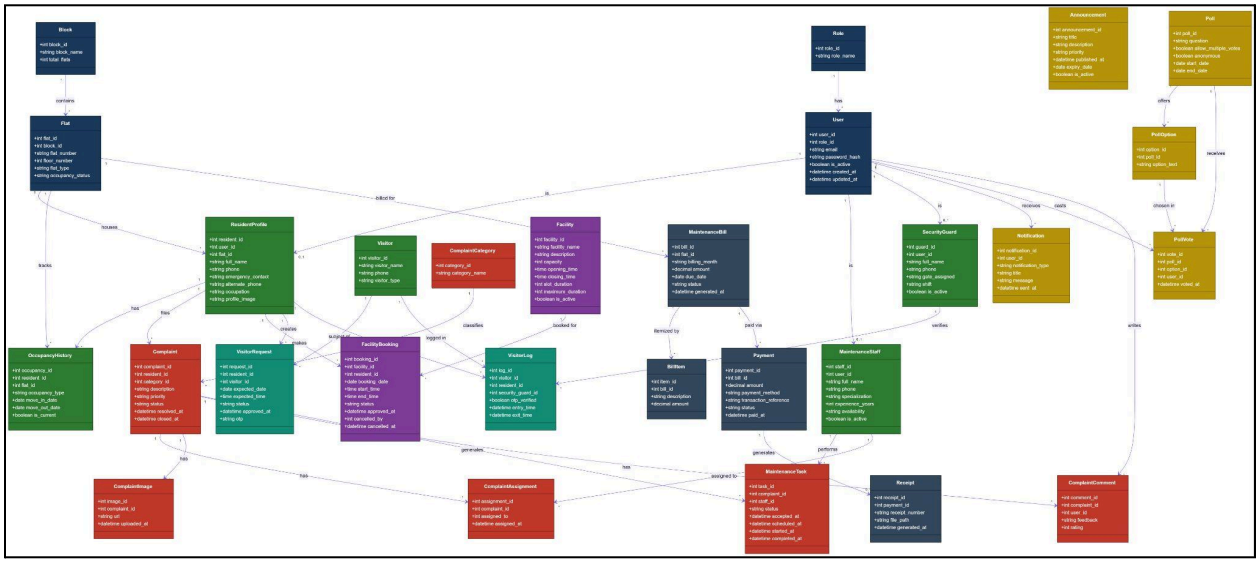
US-15	Vendor visit is verified before entry		
US-16 to US-20: AI-assisted complaint priority recommendation, staff recommendations, aggregated complaint summaries, complaint classification and community chatbot (Added later)			

---

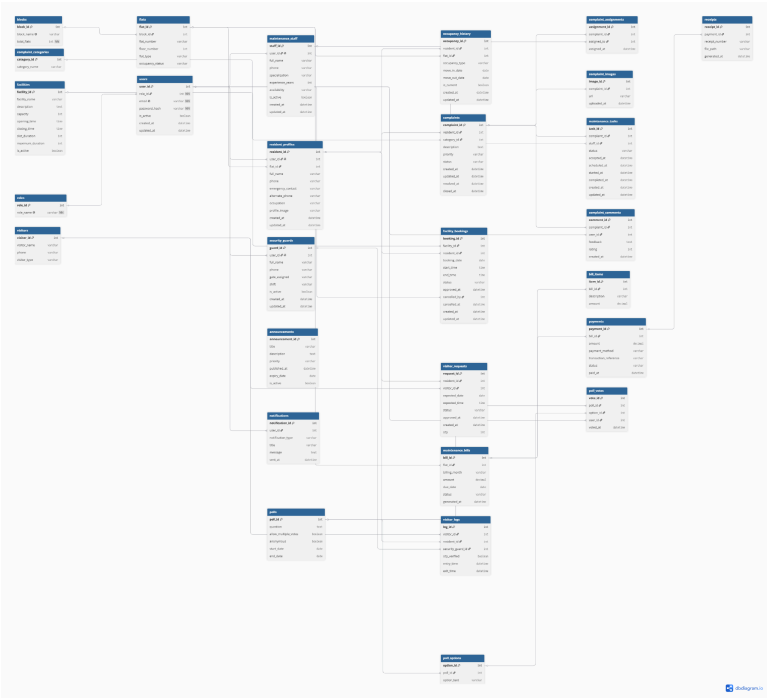
3. System Design

3.1 System Architecture

Neighborly follows a modular web application architecture in which the frontend communicates with the backend through REST APIs. The backend manages authentication, business logic, database operations, and integrations such as AI services.



[Click here to view full detailed Database Design diagram]



[Click here to view full detailed ER diagram]

The major components of the system are:

Component	Responsibility
Frontend	Provides role-specific interfaces for residents, administrators, maintenance staff and security staff
FastAPI Backend	Handles REST API requests, business logic and application workflows
Authentication & RBAC	Provides JWT-based authentication and role-based access control
Database	Stores users, apartments, complaints, bookings, visitors, bills, notifications and other system data
AI Services	Provides complaint classification, staff recommendations, smart search, reports and community chat
Testing Layer	Validates API behaviour, business rules, authorization and error handling

3.2 Major System Modules

The final implementation consists of 10 backend modules covering the core apartment-management workflows:

Module	Major Functionality
1. Authentication & User Management	Registration, login, verification, user management and profile handling
2. Apartment Management	Blocks, flats, residents and occupancy management
3. Complaint Management	Complaint creation, assignment, status tracking, comments and images
4. Maintenance Staff Management	Task assignment and complete maintenance task lifecycle
5. Visitor Management	Pre-registration, OTP verification and entry/exit logging
6. Facility Management	Facility administration, booking and conflict detection
7. Communication & Notifications	Announcements and resident notifications
8. Billing & Payments	Bills, payments, receipts and billing summaries
9. Community Engagement	Poll creation, voting and live results
10. AI Community Intelligence	Complaint classification, staff recommendations, search, reports and chatbot

---

4. Implementation

4.1 Technologies and Tools Used

Category	Technologies / Tools
Frontend	React, Vite
Backend	Python, FastAPI

Database	SQLite, SQLAlchemy
Authentication	JWT, RBAC
API Documentation	Swagger / OpenAPI 3.1.0
Testing	Pytest, HTTPX
AI Integration	Groq
Version Control	Git, GitHub
Project Management	Jira
Development	VS Code

4.2 Implemented Functionality

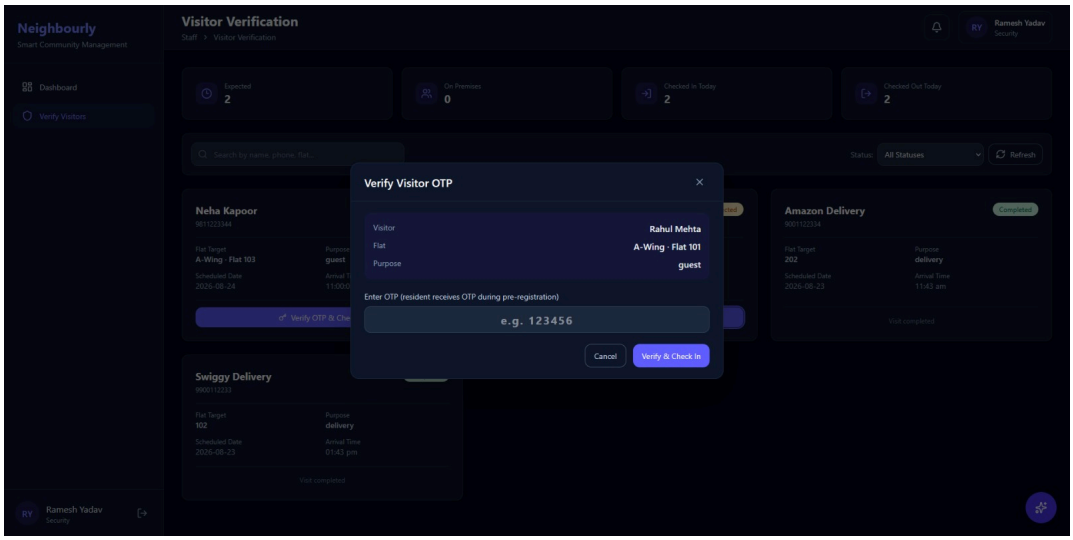
The final system provides the following major functionality:

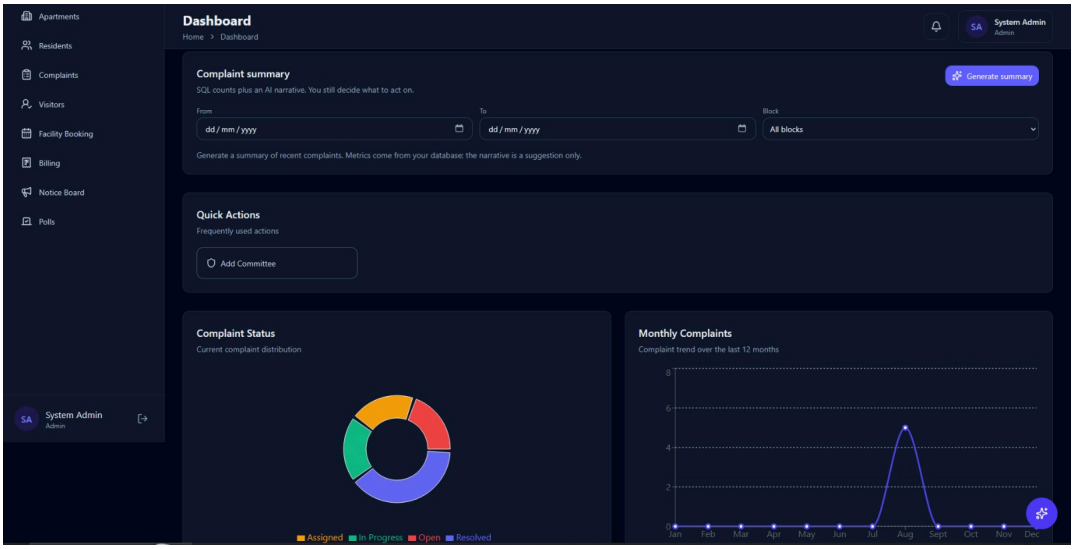
- **Authentication & User Management:** registration, login, verification, password management and role-based access.
- **Apartment Management:** management of blocks, flats, residents and occupancy history.
- **Complaint & Maintenance Management:** complaint creation, assignment, status tracking and maintenance task lifecycle.
- **Visitor Management:** visitor pre-registration, OTP verification and entry/exit logging.
- **Facility Booking:** facility management, booking, conflict detection and cancellation.
- **Communication:** announcements and centralized notifications.
- **Billing & Payments:** bill generation, payment recording, receipts and billing summaries.
- **Community Engagement:** polls, voting and live results.
- **AI Intelligence:** complaint classification, staff recommendations, smart search, complaint summaries and AI chatbot.

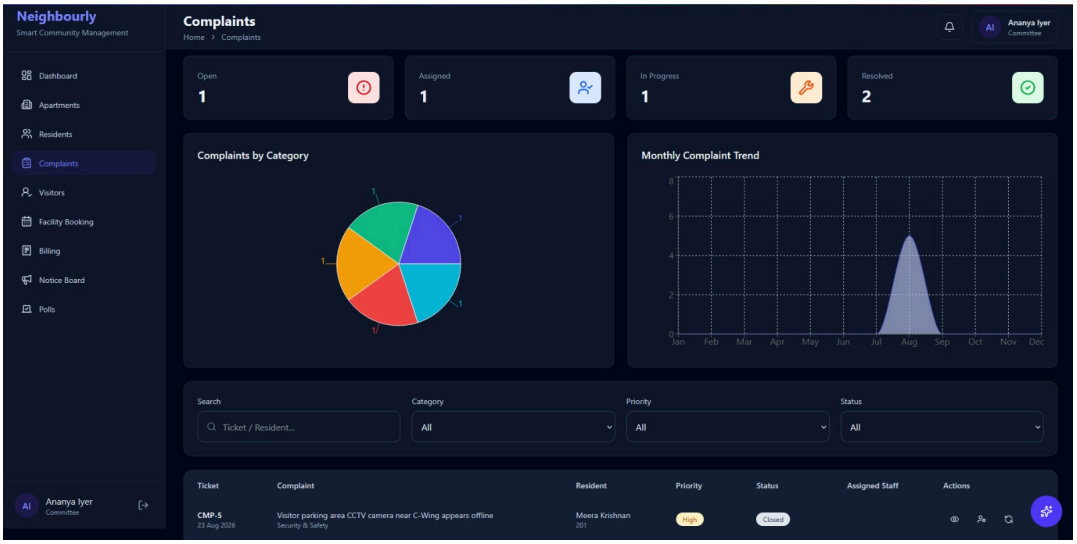
4.3 Final Implementation Status

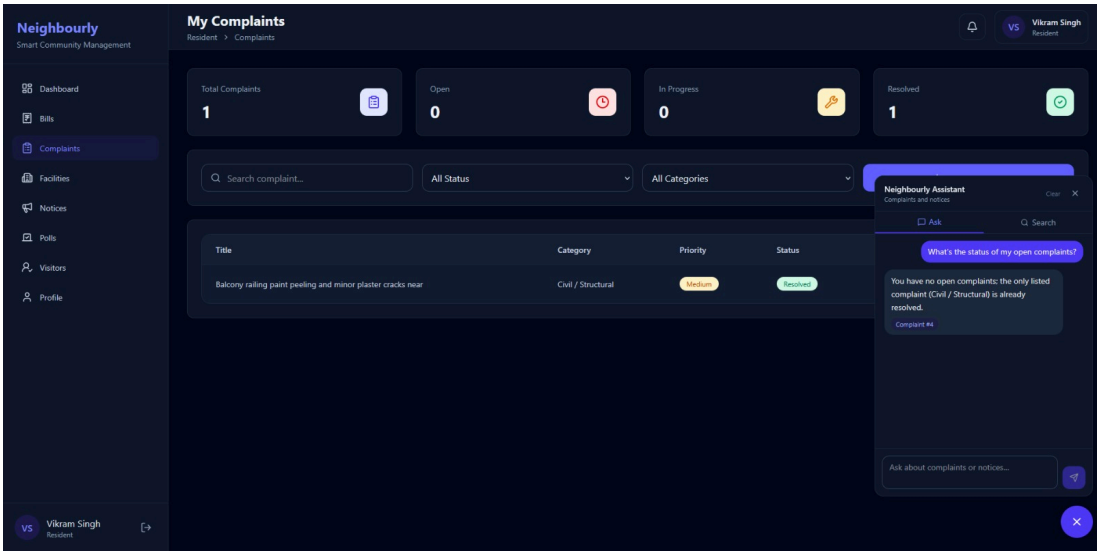
The final backend consists of 97 API endpoints across 10 modules. The implementation includes authentication and RBAC, input validation, business-rule enforcement and

role-scoped operations. AI functionality is integrated into selected complaint, search, reporting and community workflows.









5. Testing & Validation

5.1 Testing Strategy

The testing strategy combines direct pytest validation with contract-based verification to ensure API correctness, security, and alignment with user stories.

The system was tested using an asynchronous HTTP client against the actual FastAPI application, with an isolated SQLite test database and seeded test data. Testing covered functional behaviour, RBAC, input validation, business rules, and error handling.

5.2 Test Coverage

Metric	Value

Total API Endpoints	97
Modules Implemented	10
Total Test Cases	103
Passing Test Cases	103
Failing Test Cases	0
Unverified Operations	0
Direct pytest Coverage	103 tests across 12 test files
Swagger/OpenAPI Specification	OpenAPI 3.1.0
OpenAPI Paths	78

5.3 Test Distribution

Test File	Module	Test Cases	Focus
test_auth.py	Authentication	17	Login, signup, RBAC, verification
test_apartments.py	Apartments	15	Blocks, flats, residents
test_complaints.py	Complaints	10	File, assign, comments, categories
test_maintenance.py	Maintenance	8	Task lifecycle and ownership
test_visitors.py	Visitors	7	Registration, OTP, entry/exit
test_facilities.py	Facilities	12	Booking and conflicts
test_billing.py	Billing	12	Bills, payments, receipts
test_communication.py	Communication	8	Announcements, notifications
test_community.py	Community	7	Polls and voting
test_ai.py	AI	6	Classification, recommendations, search, chat
test_full_coverage.py	All	29	Endpoint coverage and edge cases

5.4 Validation Areas

Testing covered the following response and security conditions:

- **401** - Unauthorized access
- **403** - Insufficient permissions / RBAC violations
- **404** - Non-existent resources
- **409** - Duplicate or conflicting operations
- **422** - Invalid input validation
- **503** - AI service disabled/unavailable

```
## Summary

- **Total Test Cases**: 103
- **Passed**: 103
- **Failed**: 0
- **Success Rate**: 100%

All 97 endpoints across Modules 1–10 + Health check were tested for:
- Happy-path functionality
- Role-based access control (RBAC)
- Input validation
- Business rule enforcement
- Error handling and status codes

### Coverage by Module

| Module | Endpoints | Test Cases | Coverage |
|--------|-----------|------------|----------|
| Auth | 17 | 17 | 100% |
| Apartment | 15 | 15 | 100% |
| Complaints | 10 | 10 | 100% |
| Maintenance | 8 | 8 | 100% |
| Visitor | 6 | 7 | 100%+ |
| Facility | 8 | 12 | 100% |
| Communication | 8 | 8 | 100% |
| Billing | 12 | 12 | 100% |
| Community | 6 | 7 | 100%+ |
| AI | 6 | 6 | 100% |
| Health | 1 | 1 | 100% |
| **Total** | **97** | **103** | **100%** |

Note: Some modules have more test cases than endpoints due to additional negative-path and RBAC tests.
```

---

6. User Feedback & Improvements

6.1 Feedback Collected

Feedback was collected from the key stakeholders of the system, particularly residents, administrators/committee members, and maintenance staff.

Role	Feedback Area	Key Feedback
Resident	Payment tracking	Wanted to view bills and pay them online
Resident	Notifications	Wanted a single place for complaint, booking and visitor updates

Admin	Announcements	Needed a way to publish notices to all residents
Admin	Maintenance	Needed a structured workflow for staff task management
Resident	Community engagement	Wanted to participate in community decisions through voting
Admin/Committee	AI assistance	Wanted smart suggestions for complaint categories and staff assignment

6.2 Feedback-Driven Improvements

The feedback was used to guide the development of the additional functionality implemented during Milestone 4.

Feedback	Implementation	Status
Payment tracking	Billing, payments, receipts and billing summaries	Complete
Unified notifications	Notification inbox and announcement system	Complete
Announcements	Create, update, delete and resident notifications	Complete
Maintenance flow	Accept, decline, schedule, start and complete tasks	Complete
Community engagement	Polls, voting and live results	Complete
AI assistance	Classification, staff recommendations, search, reports and chat	Complete
Complaint tracking	Enforced complaint state transitions	Complete
Visitor security	HMAC-SHA256 OTP protection	Complete
Facility conflicts	Overlap and duration validation	Complete

6.3 User Evaluation

The implemented features received positive overall feedback, with particularly strong ratings for complaint and visitor management.

Feature	Rating/5
Complaint Management	5
Facility Booking	4
Visitor Management	5
Notifications	4
AI Classification	4
Admin Dashboard	4
Billing	3
Community Polls	4

The feedback also identified areas for future enhancement. Real payment-gateway integration and SMS/email-based notifications remain pending, while the current system supports manual payment recording and an in-app notification mechanism.

---

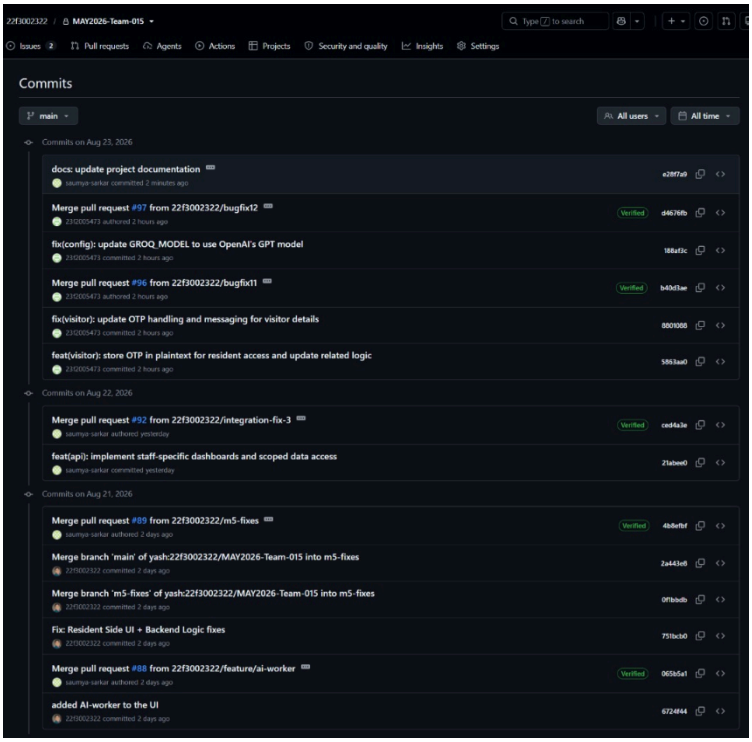
7. Code Review & Issue Tracking

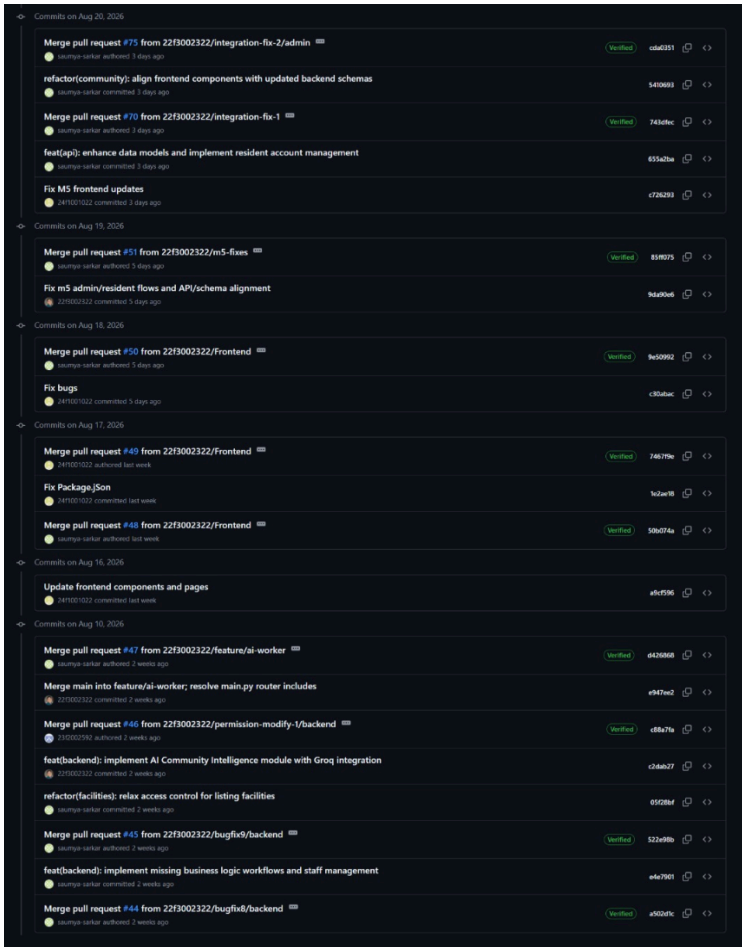
7.1 Version Control and Development Tracking

GitHub was used as the primary version-control platform throughout the project. Development was carried out feature-wise, with the team regularly updating the repository as individual features and modules were implemented.

The repository provided a centralized record of development progress and enabled the team to track changes throughout the project lifecycle.

Click here to see full commit history


22f3002322 / MAY2026-Team-015
Type / to search
Issues 2
Pull requests
Agents
Actions
Projects
Security and quality
Insights
Settings
Commits
main
All users
All time
Commits on Aug 23, 2026
docs: update project documentation
saumya-sarkar committed 2 minutes ago
e2ff7a9
Merge pull request #97 from 22f3002322/bugfix12
23f2005473 authored 2 hours ago
Verified
d4676fb
fix(config): update GROQ_MODEL to use OpenAI's GPT model
23f2005473 committed 2 hours ago
188af3c
Merge pull request #96 from 22f3002322/bugfix11
23f2005473 authored 2 hours ago
Verified
b40d3ae
fix(visitor): update OTP handling and messaging for visitor details
23f2005473 committed 2 hours ago
8001088
feat(visitor): store OTP in plaintext for resident access and update related logic
23f2005473 committed 2 hours ago
5863aa0
Commits on Aug 22, 2026
Merge pull request #92 from 22f3002322/integration-fix-3
saumya-sarkar authored yesterday
Verified
ced4a3e
feat(api): implement staff-specific dashboards and scoped data access
saumya-sarkar committed yesterday
21abee0
Commits on Aug 21, 2026
Merge pull request #89 from 22f3002322/m5-fixes
saumya-sarkar authored 2 days ago
Verified
4b8efbf
Merge branch 'main' of yash:22f3002322/MAY2026-Team-015 into m5-fixes
22f3002322 committed 2 days ago
2a443e8
Merge branch 'm5-fixes' of yash:22f3002322/MAY2026-Team-015 into m5-fixes
22f3002322 committed 2 days ago
0f1bbdb
Fix: Resident Side UI + Backend Logic fixes
22f3002322 committed 2 days ago
751bcb0
Merge pull request #88 from 22f3002322/feature/ai-worker
saumya-sarkar authored 2 days ago
Verified
065b5a1
added AI-worker to the UI
22f3002322 committed 2 days ago
6724f44


Commits on Aug 20, 2026
Merge pull request #75 from 22f3002322/integration-fix-2/admin
saumya-sarkar authored 3 days ago
Verified
cda0351
refactor(community): align frontend components with updated backend schemas
saumya-sarkar committed 3 days ago
5410693
Merge pull request #70 from 22f3002322/integration-fix-1
saumya-sarkar authored 3 days ago
Verified
743dfec
feat(api): enhance data models and implement resident account management
saumya-sarkar committed 3 days ago
655a2ba
Fix M5 frontend updates
24f1001022 committed 3 days ago
c726293
Commits on Aug 19, 2026
Merge pull request #51 from 22f3002322/m5-fixes
saumya-sarkar authored 5 days ago
Verified
85ff075
Fix m5 admin/resident flows and API/schema alignment
22f3002322 committed 5 days ago
9da90e6
Commits on Aug 18, 2026
Merge pull request #50 from 22f3002322/Frontend
saumya-sarkar authored 5 days ago
Verified
9e50992
Fix bugs
24f1001022 committed 5 days ago
c80abac
Commits on Aug 17, 2026
Merge pull request #49 from 22f3002322/Frontend
24f1001022 authored last week
Verified
7467f9e
Fix Package.jSon
24f1001022 committed last week
1e2ae18
Merge pull request #48 from 22f3002322/Frontend
saumya-sarkar authored last week
Verified
50b074a
Commits on Aug 16, 2026
Update frontend components and pages
24f1001022 committed last week
a6cf596
Commits on Aug 10, 2026
Merge pull request #47 from 22f3002322/feature/ai-worker
saumya-sarkar authored 2 weeks ago
Verified
d426868
Merge main into feature/ai-worker; resolve main.py router includes
22f3002322 committed 2 weeks ago
e947ee2
Merge pull request #46 from 22f3002322/permission-modify-1/backend
23f2002592 authored 2 weeks ago
Verified
c88a7fa
feat(backend): implement AI Community Intelligence module with Groq integration
22f3002322 committed 2 weeks ago
c2dab27
refactor(facilities): relax access control for listing facilities
saumya-sarkar committed 2 weeks ago
05f28bf
Merge pull request #45 from 22f3002322/bugfix9/backend
saumya-sarkar authored 2 weeks ago
Verified
522e98b
feat(backend): implement missing business logic workflows and staff management
saumya-sarkar committed 2 weeks ago
e4e7901
Merge pull request #44 from 22f3002322/bugfix8/backend
saumya-sarkar authored 2 weeks ago
Verified
a502d1c

7.2 Issue Reporting and Resolution

Issues identified during implementation and testing were recorded and tracked until resolution. Testing played an important role in identifying implementation and data-isolation issues that were not apparent during initial development.

The major issues identified and resolved included:

[Click here to see all issues raised and closed]



7.3 Code Review

Code review was performed as part of the team's collaborative development process to verify feature implementation, identify issues, and maintain consistency across modules. Each member of the team reviewed all features and components. Changes were reviewed alongside testing before being considered complete.

---

8. Milestones & Sprint Summary

8.1 Detailed Milestone Schedule

Overall Project Schedule

Milestone	Phase	Duration	Deliverables
Milestone 1	User Requirement Analysis	15 Jun 2026 – 28 Jun 2026	User identification, stakeholder interviews, pain point analysis, SMART user stories, Product Backlog
Milestone 2	Planning & System Design	29 Jun 2026 – 22 Jul 2026	Project schedule, Sprint schedule, Gantt Chart, Kanban Board, Component Design, Class Diagram, ER Diagram, Frontend Prototype, Documentation
Milestone 3	Backend Development	23 Jul 2026 – 02 Aug 2026	Authentication module, Complaint APIs, Visitor APIs, Booking APIs, Billing APIs, Notification APIs, Swagger Documentation, Backend Testing
Milestone 4	Integration & System Testing	03 Aug 2026 – 12 Aug 2026	Frontend-Backend Integration, System Testing, Bug Fixes, UI Refinements, Sprint Review & Retrospective
Milestone 5	Final Submission	13 Aug 2026 – 23 Aug 2026	Final Documentation, README, Code Packaging, Presentation, Demo

			Video, Final Submission

Milestone - Wise Schedule

Milestone 1 - User Requirement Analysis		
Activity	**Expected Output**	**Target Date**
Identify stakeholders	List of primary, secondary, and tertiary users	16 Jun 2026
Conduct stakeholder interviews	Interview observations and user feedback	20 Jun 2026
Analyze user pain points	Documented problems and requirements	22 Jun 2026
Prepare SMART User Stories	Prioritized user stories	25 Jun 2026
Create Product Backlog	Initial product backlog	27 Jun 2026
Compile Milestone 1 Report	Completed Milestone 1 documentation	28 Jun 2026

Milestone 2 - Planning & System Design		
Activity	**Expected Output**	**Target Date**
Finalize project architecture	High-level system architecture	09 Jul 2026
Design system components	Component Design document	10 Jul 2026
Prepare Class Diagram	UML Class Diagram	12 Jul 2026
Prepare ER Diagram	Entity-Relationship Diagram	12 Jul 2026
Develop Sprint Schedule	Sprint planning document	07 Jul 2026
Create Gantt Chart	Project timeline	07 Jul 2026
Configure Jira & Kanban Board	Scrum board with backlog and sprints	07 Jul 2026

Develop frontend prototype	UI screens and navigation flow	18 Jul 2026
Compile Milestone 2 Report	Complete planning and design documentation	22 Jul 2026

Milestone 3 - Backend Development		
Activity	**Expected Output**	**Target Date**
Authentication & RBAC	Secure login and authorization module	24 Jul 2026
User & Apartment Management	User and apartment APIs	26 Jul 2026
Complaint Management	Complaint handling APIs	28 Jul 2026
Visitor & Facility Management	Visitor and booking APIs	30 Jul 2026
Billing & Notifications	Payment and notification modules	31 Jul 2026
API Documentation	Swagger/OpenAPI documentation	01 Aug 2026
Backend Testing	Tested and validated APIs	02 Aug 2026

Milestone 4 - Integration & System Testing		
Activity	**Expected Output**	**Target Date**
Frontend-Backend Integration	Fully integrated application	05 Aug 2026
Functional Testing	Verified application functionality	07 Aug 2026
Bug Resolution	Stable and error-free system	09 Aug 2026
UI Improvements	Enhanced user experience	10 Aug 2026
Sprint Review	Reviewed sprint deliverables	11 Aug 2026
Sprint Retrospective	Identified improvements for project completion	12 Aug 2026

Milestone 5 - Final Submission		
Activity	**Expected Output**	**Target Date**
Final Documentation	Complete project report	17 Aug 2026
README & Installation Guide	Deployment instructions	18 Aug 2026
Code Packaging	Submission-ready source code	19 Aug 2026
Presentation Preparation	Final presentation slides	20 Aug 2026
Demo Video	Recorded project demonstration	21 Aug 2026
Final Verification	Reviewed and validated deliverables	22 Aug 2026
Project Submission	Successful submission of all deliverables	23 Aug 2026

8.2 Detailed Sprint Schedule

Sprint 0: Milestone 1 - User Requirement Analysis

Objectives:

- Identify the primary, secondary, and tertiary users of the system.
- Conduct stakeholder interviews and gather user requirements.
- Analyze user pain points and existing apartment management challenges.
- Formulate SMART user stories based on stakeholder needs.
- Prioritize user stories to establish the initial product backlog.
- Prepare and submit the Milestone 1 documentation.

Timeline: 15/06/2026 – 28/06/2026

Sprint 1: Milestone 2 - Planning & System Design

Objectives:

- Finalize project scope, branding, technology stack, and development tools.
- Prepare the project schedule, sprint schedule, and Gantt chart.
- Configure the Jira project, Kanban board, Epics, Stories, and Sprint Backlog.
- Design the system architecture and functional components.
- Develop the Class Diagram and Entity-Relationship (ER) Diagram.
- Design and prototype all major frontend pages using Figma.

- Prepare the Milestone 2 report with all required design artifacts and screenshots.

Timeline: 04/07/2026 – 22/07/2026

Sprint 2: Backend Development & API Implementation

Objectives:

- Develop backend modules using FastAPI.
- Implement Authentication and Role-Based Access Control (RBAC).
- Develop APIs for complaints, visitor management, facility booking, billing, announcements, and community engagement.
- Prepare OpenAPI/Swagger documentation for all APIs.
- Perform backend testing using Pytest.
- Review completed APIs and incorporate feedback before integration.

Timeline: 23/07/2026 – 02/08/2026

Sprint 3: Frontend Integration & System Testing

Objectives:

- Integrate frontend pages with backend APIs.
- Implement complete end-to-end user workflows.
- Perform functional, integration, and system testing.
- Resolve bugs and optimize user experience.
- Collect user feedback and perform final UI refinements.
- Conduct Sprint Review and Sprint Retrospective.

Timeline: 03/08/2026 – 12/08/2026

Final Submission Sprint

Objectives:

- Perform final application testing and validation.
- Complete project documentation and report compilation.
- Prepare presentation slides and demonstration materials.
- Package the source code with installation instructions and README.
- Verify all deliverables before submission.
- Submit the final report, source code, presentation, and demonstration video.

Timeline: 13/08/2026 – 23/08/2026

8.3 Milestone Summary

Milestone	Focus	Major Outcomes
Milestone 1	User Requirements	Identified stakeholders, conducted user research, documented pain points and developed user stories
Milestone 2	System Design & Planning	Finalized system architecture, database design, component structure, project schedule and development plan
Milestone 3	Backend Implementation	Implemented core backend modules and APIs, followed by API testing and validation
Milestone 4	Feature Expansion & Improvement	Added billing, communication, community engagement, maintenance management and AI capabilities based on user feedback
Milestone 5	Final Submission	Consolidated the completed system, final testing, documentation, working prototype and project deliverables

8.4 Sprint Summary

The development work was divided into sprints, with each sprint focusing on implementing and validating a specific set of system functionality.

Sprint	Major Work
Sprint 1	Implemented the initial backend functionality and core APIs required for the primary apartment-management workflows.
Sprint 2	Extended the backend with additional management workflows and completed testing of the implemented APIs.
Sprint 3	Incorporated user feedback and developed additional modules including billing, communication, maintenance workflows and community engagement.
Sprint 4	Integrated AI-assisted functionality, completed feature-level testing, fixed identified issues and refined the overall system.
Final Phase	Performed final validation, consolidated project documentation and prepared the working prototype and final submission deliverables.

8.5 Development Approach

Across the milestones and sprints, the team followed an iterative development approach. Features were implemented incrementally, tested against their expected behaviour, reviewed, and refined based on identified issues and user feedback. This allowed the system to evolve from the initial requirements and design into the final working implementation.

---

9. Final System Status & Conclusion

9.1 Final System Status

At the end of the project, Neighborly provides a working apartment association management platform covering resident, administration, maintenance and security workflows, along with billing, community engagement and AI-assisted functionality.

Metric	Final Status
API Endpoints	97
Modules Implemented	10
Total Test Cases	103
Passing Test Cases	103
Failing Test Cases	0
Unverified Operations	0
Direct pytest Coverage	103 tests across 12 test files
Swagger/OpenAPI	OpenAPI 3.1.0
User Stories Covered	US-01 through US-20
Working Prototype	Completed

9.2 Key Achievements

The final implementation successfully provides:

- Centralized apartment and resident management.

- Complaint filing, assignment and lifecycle tracking.
- Structured maintenance staff workflows.
- Visitor pre-registration and secure OTP verification.
- Facility booking with conflict and duration validation.
- Announcements and centralized notifications.
- Billing, payment recording and receipt generation.
- Community polls and resident voting.
- AI-assisted complaint classification and staff recommendations.
- AI-powered search, complaint reports and community chat.
- Authentication and role-based access control across system operations.
- Comprehensive automated API validation with 103/103 tests passing.

9.3 Limitations & Future Scope

While the major planned functionality has been implemented, some enhancements remain for future development:

- Integration with a real online payment gateway.
- SMS and email notifications in addition to in-app notifications.
- Further enhancement of AI capabilities and accuracy.
- Additional dashboard filtering and reporting capabilities.
- Further performance and scalability optimization for larger apartment communities.

9.4 Conclusion

Neighborly was developed as a centralized solution to address the fragmented processes commonly involved in apartment association management. The project progressed from user research and requirements gathering through system design, iterative implementation, testing, user feedback and feature enhancement.

The final system demonstrates the integration of web technologies, role-based security, automated testing, community-management workflows and AI-assisted functionality into a single working platform. With all 103 automated test cases passing and 0 unverified operations, the implementation provides a validated foundation that can be extended with additional integrations and capabilities in future iterations.

Milestone 1 | Milestone 2 | Milestone 3 | Milestone 4 | Milestone 5 | Git Repo | Code Base

--- End of Milestone 5 Report ---